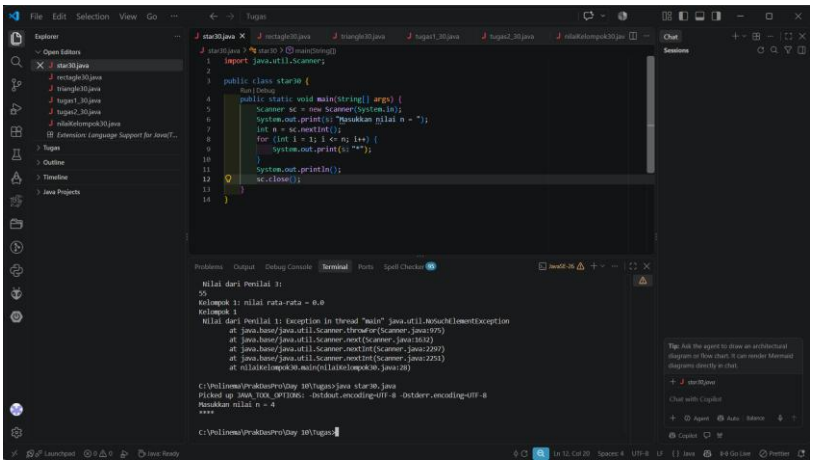
Jobsheet 08

Nama	Viqi Ahmad Ridwan
Kelas	TI-1C
Absen	30
NIM	264107020196

1.1. Percobaan 1: Review Perulangan 1



- Jika inisialisasi i = 1 diubah menjadi i = 0 tanpa mengubah kondisi, berapa karakter * yang tercetak ketika n = 5? Jelaskan.
- Jika kondisi i <= n diganti menjadi i < n, berapa kali perulangan dijalankan saat n = 5? Mengapa hasilnya berbeda?
- Prediksi output potongan kode berikut ketika n = 9, kemudian buktikan dengan mengganti perulangan FOR pada program Anda dan menjalankannya.

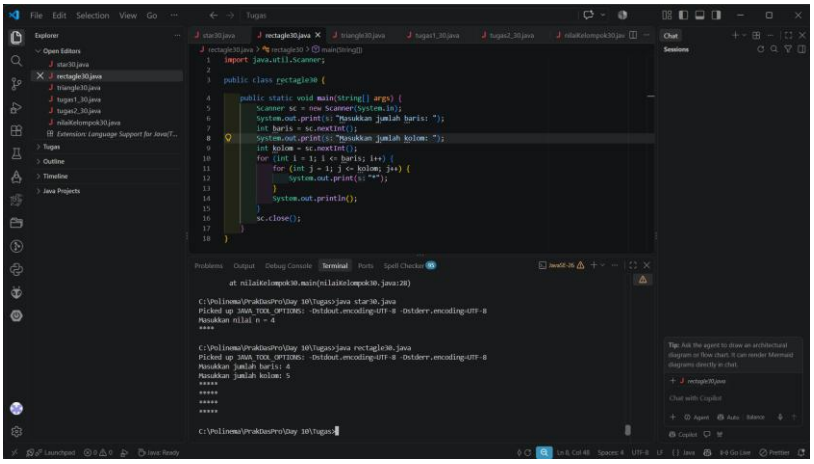
  ```
  for (int i = 2; i <= n; i += 2) {
      System.out.print(i + " ");
  }
  ```

  Modifikasi kode tersebut agar mencetak bilangan ganjil positif yang tidak melebihi n.
- Jelaskan mengapa perubahan increment i++ menjadi i-- (dengan kondisi tetap i <= n) membuat perulangan berjalan terus-menerus dan tidak berhenti dalam waktu wajar untuk n positif. Jangan menjalankan versi tersebut; jika terlanjur berjalan, hentikan program dengan menekan Ctrl + C pada terminal.

- Jika i = 0 dan kondisi tetap i <= n, ketika n = 5, karakter * akan tercetak sebanyak 6 kali. Nilai i yang diproses adalah 0, 1, 2, 3, 4, dan 5.

- Ketika n = 5, perulangan dijalankan sebanyak 4 kali. Kondisi i < 5 hanya benar untuk i = 1, 2, 3, 4.
- Output : 2 4 6 8
  Modifikasi : nilai I = 1 , output : 1 3 5 7 9
  ```
  for (int i = 1; i <= n; i += 2) {
      System.out.print(i + " ");
  }
  System.out.println();
  ```
- Jika i++ diubah menjadi i-- sementara kondisi tetap i <= n dan n positif, perulangan akan berjalan terus-menerus. Setiap perulangan, nilai i berkurang satu, nilai i menjadi 1, 0, -1, -2, -3, dan seterusnya.

1.2. Percobaan 2: Pola Bintang Persegi Panjang (Nested FOR)

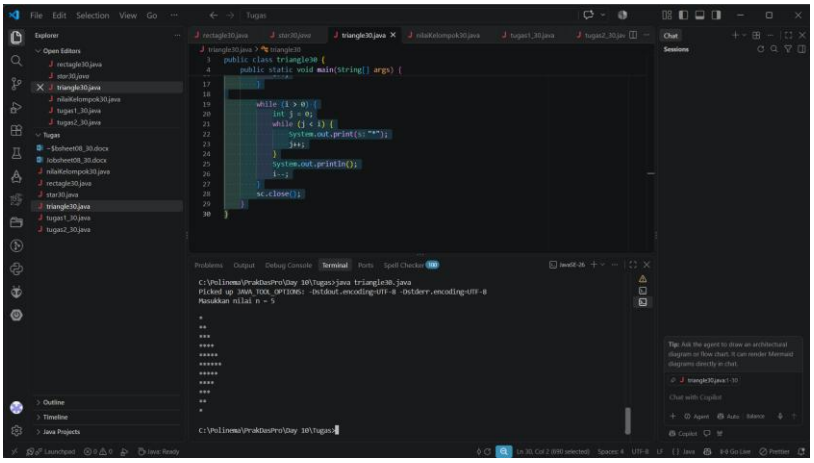


- Tentukan fungsi variabel i dan j pada program tersebut.
- Berapa kali outer loop, inner loop, dan perintah print("*") dijalankan untuk baris = 4 dan kolom = 7?
- Apa yang terjadi jika batas perulangan luar dan dalam ditukar (outer menggunakan kolom, inner menggunakan baris)? Gunakan baris = 3 dan kolom = 6 untuk menjelaskan.
- Jelaskan akibat penghapusan System.out.println(); setelah inner loop.
- Modifikasi program agar karakter yang dicetak adalah #, lalu commit dan push perubahan ke GitHub.

- Variabel i berfungsi sebagai penghitung perulangan luar (outer loop) yang menentukan jumlah baris yang akan dicetak.
  Variabel j berfungsi sebagai penghitung perulangan dalam (inner loop) yang menentukan jumlah kolom atau karakter * yang dicetak pada setiap baris.
- Outer loop: dijalankan sebanyak 4 kali karena jumlah baris adalah 4.
  Inner loop: dijalankan sebanyak 28 kali secara keseluruhan, yaitu 7 kali untuk setiap baris × 4 baris.

- Jika outer loop menggunakan kolom dan inner loop menggunakan baris, ketika baris = 3 dan kolom = 6, program akan mencetak 6 baris dengan masing-masing 3 karakter *.
  outputnya :
  ***
  ***
  ***
  ***
  ***
  ***
- Jika System.out.println(); dihapus setelah inner loop, karakter * tidak akan berpindah ke baris baru setelah satu baris selesai dicetak. Akibatnya, semua karakter akan tercetak dalam satu baris.
- System.out.print("#");
  output jika baris = 4 dan kolom = 7 :
  #######
  #######
  #######
  #######

1.3. Percobaan 3: Debugging Bintang Segitiga – Nested WHILE



- Sebutkan ketidaksesuaian output awal dan minimal dua perbaikan yang diperlukan untuk menghasilkan pola segitiga bertingkat.
- Jelaskan peran variabel i dan j. Mengapa nilai j harus diinisialisasi kembali ke 0 pada setiap iterasi outer loop? Apa yang terjadi jika j tidak di-reset?
- Jelaskan perubahan inisialisasi, kondisi, dan perubahan nilai (update) pada outer loop yang diperlukan untuk membentuk segitiga terbalik.

- Nilai awal i = 0 menyebabkan perulangan pertama tidak mencetak karakter *, karena kondisi j < i bernilai 0 < 0 atau salah.
  Tidak ada System.out.println() setelah inner loop, sehingga semua karakter * dicetak dalam satu baris, bukan bertingkat.

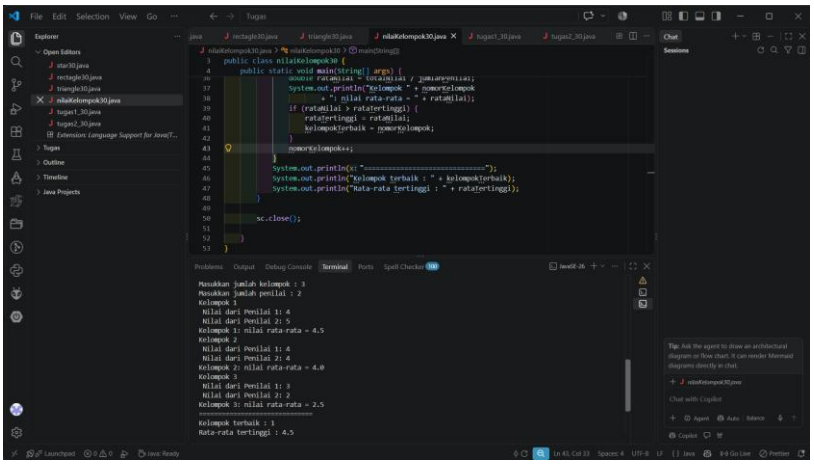
Perbaikan
Mengubah int i = 0; menjadi int i = 1; agar baris pertama mencetak satu karakter *.
Menambahkan System.out.println(); setelah inner loop agar setiap baris berpindah ke baris baru.

- Variabel i berfungsi sebagai penghitung perulangan luar (outer loop) yang menentukan jumlah baris segitiga.
  Variabel j berfungsi sebagai penghitung perulangan dalam (inner loop) yang menentukan jumlah karakter * pada setiap baris.

  Variable j harus di di inisialisasi ke 0
  Karena setiap baris harus memulai penghitung karakter dari nol. Dengan begitu, jumlah * dapat disesuaikan dengan nilai i pada baris tersebut.
  Inisialisasi i = n: memulai dari jumlah bintang terbanyak.

- Kondisi i > 0: perulangan berlanjut selama nilai i masih lebih besar dari nol.
  Update i--: mengurangi jumlah bintang sebanyak satu pada setiap baris.

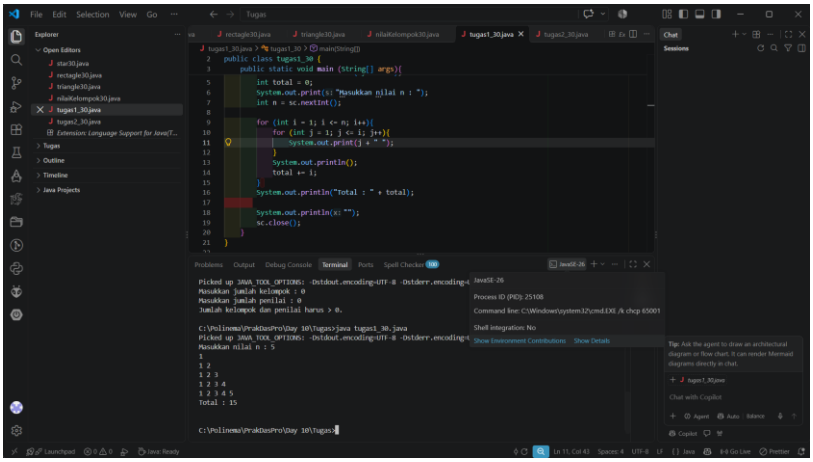
1.4. Percobaan 4: Studi Kasus Nilai Tugas Proyek Kelompok



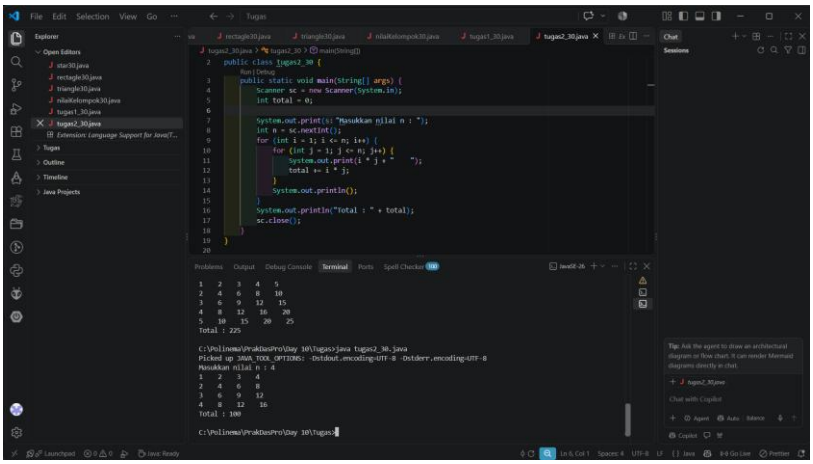
- Mengapa totalNilai perlu di-reset di dalam outer loop, tetapi rataTertinggi tidak boleh di-reset pada setiap kelompok?
- Untuk 3 kelompok dan 2 penilai (tanpa input yang salah), berapa kali badan perulangan FOR dijalankan secara keseluruhan?
- Apa yang terjadi jika totalNilai dideklarasikan sebagai int dan rata-rata dihitung menggunakan pembagian dua bilangan bulat? Berikan contoh nilai.
- Mengapa rataTertinggi diberi nilai awal -1, bukan 0? Berikan contoh input yang membuat hasil salah apabila nilai awalnya 0.
- Program ini memiliki tiga tingkat perulangan. Sebutkan masing-masing jenis perulangan beserta perannya

- Karena setiap kelompok memiliki total nilai masing-masing. Jika tidak di-reset, nilai kelompok sebelumnya akan ikut terhitung pada kelompok berikutnya sehingga rata-ratanya salah. rataTertinggi tidak boleh di-reset pada setiap kelompok karena variabel ini digunakan untuk menyimpan rata-rata tertinggi dari seluruh kelompok yang sudah diperiksa.
- Setiap kelompok dinilai oleh 2 penilai, sehingga jumlah seluruh eksekusi badan `FOR` adalah:
  3x2 = 6 kali perulangan
  Jadi, badan perulangan `FOR` dijalankan 6 kali secara keseluruhan, dengan asumsi tidak ada input yang salah.
- Jika totalNilai bertipe int dan rata-rata dihitung menggunakan pembagian dua bilangan bulat, hasil pecahan akan dibuang karena pembagian bilangan bulat di Java tidak menyimpan bagian desimal.
- Nilai awal `-1` digunakan agar rata-rata kelompok pertama pasti lebih besar daripada nilai awal tersebut, karena nilai yang valid berada pada rentang 0–100. Jika nilai awalnya `0`, program dapat salah menentukan kelompok terbaik apabila seluruh kelompok memperoleh rata-rata `0`.
- While – While Mengulang proses penilaian untuk setiap kelompok.
  for – for Meminta nilai dari setiap penilai pada kelompok yang sedang dinilai.
  do-while – do Meminta input nilai dan mengulang jika nilai di luar rentang 0–100.

- **Tugas :**
  - **Tugas 1**



  - **Tugas 2**



  - **Tugas 3**

